FIAP

CURSO SUPERIOR DE TECNOLOGIA EM CYBERSECURITY

CRITICAL BAIT

RELATÓRIO EXECUTIVO — CP03 DEVSECOPS

INTEGRANTES

RM	Nome completo
561961	Gabriel Beoni Sanches Sammartino
563302	Enzo Marques Mizuguchi
561794	Victor Nicolas Silva Pereira
562610	Clara Espanha Alves

São Paulo — 2026

SUMÁRIO

1 SÍNTESE EXECUTIVA 3

2 ARQUITETURA E GOVERNANÇA DA ESTEIRA 4

3 INDICADORES DE SEGURANÇA 5

4 MATRIZ DE ACHADOS E RISCOS RESIDUAIS 6

5 VALIDAÇÃO DO SECURITY GATE 7

6 RASTREABILIDADE DO DEPLOY 8

7 EVIDÊNCIAS DE PRODUÇÃO 9

Documento executivo baseado nos registros da execução #15 e na evidência de bloqueio da execução #23. As imagens reproduzidas são registros operacionais do projeto.

1 SÍNTESE EXECUTIVA

O projeto Critical Bait implementou uma esteira CI/CD com controles de segurança antes da publicação em produção. A execução de referência #15, na branch main, foi concluída com sucesso; a execução de teste #23, na branch teste-quality-gate, demonstra a interrupção da esteira quando o job inicial de segurança falha.

A arquitetura integra Gitleaks, Semgrep, Trivy, Docker/Buildx, GitHub Container Registry (GHCR), OWASP ZAP, aprovação manual e deploy no Render. Os resultados e evidências foram preservados em artefatos do GitHub Actions.

Quadro executivo de decisão

Dimensão	Resultado	Leitura executiva
Execução principal	#15: success	Fluxo completo executado, incluindo produção.
Bloqueio preventivo	#23: Failure no Gitleaks	Jobs seguintes não executados no teste.
Segurança	0 HIGH/CRITICAL no resumo	Persistem achados de menor severidade e divergências de relatório.
Entrega	11 artifacts publicados	Evidências técnicas disponíveis para auditoria.

Decisão: a execução #15 comprova a operação da esteira no escopo testado. A aprovação operacional não equivale à inexistência de vulnerabilidades: os achados residuais e a consistência entre formatos de relatório devem continuar sendo acompanhados.

2 ARQUITETURA E GOVERNANÇA DA ESTEIRA

O fluxo aplica controles shift-left antes do build e verificações adicionais sobre o artefato e a aplicação em homologação. A etapa de produção exige aprovação manual documentada.

Fase	Controle / saída	Fase	Controle / saída
01 Gitleaks	Segredos	02 Semgrep	SAST
Fase	**Controle / saída**	**Fase**	**Controle / saída**
03 Build	Empacotamento	04 Trivy SCA	Dependências / Dockerfile
Fase	**Controle / saída**	**Fase**	**Controle / saída**
05 Docker + GHCR	Imagem por SHA	06 Trivy Container	Scan de imagem
Fase	**Controle / saída**	**Fase**	**Controle / saída**
07 Staging + ZAP	DAST	08A Preparação	GitHub Pages
Fase	**Controle / saída**	**Fase**	**Controle / saída**
08B Aprovação + deploy	Gate humano / Render	09 Resumo	Consolidação

O fluxo operacional é sequencial em seus gates relevantes; a consolidação final reúne os resultados dos jobs e os artifacts. O uso de tag por commit permite relacionar código-fonte e imagem construída.

Ponto de controle: a aprovação manual do ambiente de produção ocorre antes do job de deploy, conforme as evidências coletadas.

3 INDICADORES DE SEGURANÇA

Controle	Resultado observado	Interpretação
Gitleaks	0 segredos	Nenhuma ocorrência no relatório da execução #15.
Semgrep	0 achados / 0 erros	Sem achados nas regras executadas.
Trivy SCA	0 HIGH / CRITICAL	TXT aponta uma configuração LOW; JSON diverge.
Trivy Container	0 HIGH / CRITICAL	TXT aponta 2 UNKNOWN e 1 MEDIUM; JSON diverge.
OWASP ZAP	66 PASS / 0 FAIL / 2 WARN	Dois tipos de alerta informativo no baseline.
Produção	Jobs 08A e 08B: success	Publicação em Pages e Render.
Artifacts	11 publicados	Registros da execução preservados.

Os indicadores refletem os relatórios e o resumo fornecidos para a execução #15. Zero achados HIGH/CRITICAL não implica ausência de vulnerabilidades, especialmente quando há achados de outras severidades ou divergência entre formatos de saída.


09 - Resumo da Esteira summary

Critical Bait - Resumo da Esteira

Commit: 626d0a2b6bc5c75df4a87457ae7c712256314219

Branch: main

Execucao: 15

Etapa	Resultado	Achados
01 - Gitleaks	success	0 segredos
02 - Semgrep	success	0 achados
03 - Build	success	N/A
04 - Trivy SCA	success	0 HIGH/CRITICAL
05 - Docker e GHCR	success	N/A
06 - Trivy Container	success	0 HIGH/CRITICAL
07 - Staging e ZAP	success	Alto: 0; Medio: 0; Baixo: 0; Informativo: 2
08A - Preparar producao	success	N/A
08B - Deploy producao	success	N/A

Figura 1 — Resumo consolidado da execução #15. Fonte: GitHub Actions, 2026.

4 MATRIZ DE ACHADOS E RISCOS RESIDUAIS

Origem	Achado / severidade	Impacto e encaminhamento
Trivy SCA	DS-0026: HEALTHCHECK ausente (LOW)	Adicionar HEALTHCHECK e reexecutar análise.
Trivy Container	musl e musl-utils (UNKNOWN)	Investigar classificação e atualizar base/imagem quando aplicável.
Trivy Container	zlib (MEDIUM), CVE-2026-85091	Verificar correção disponível e atualizar pacote/imagem.
OWASP ZAP	Non-Storable Content (informativo)	Revisar política de cache conforme conteúdo exposto.
OWASP ZAP	Modern Web Application (informativo)	Registrar característica da aplicação; avaliar necessidade de testes autenticados.
Relatórios Trivy	TXT e JSON com contagens distintas	Revisar parâmetros, versões e escopo; manter ambos como evidência.

Limitação: no Trivy SCA, o TXT registra 27 testes (26 sucessos, 1 falha LOW), enquanto o JSON registra 20 verificações e zero falhas. No Trivy Container, o TXT contém três ocorrências não reproduzidas no JSON. A origem exata das diferenças não foi comprovada pelos registros disponíveis.

O resultado do OWASP ZAP corresponde à varredura baseline realizada em homologação; não substitui testes aprofundados de autenticação, autorização e lógica de negócio.

5 VALIDAÇÃO DO SECURITY GATE

A execução de teste #23, na branch teste-quality-gate, terminou com status Failure. O job 01 — Gitleaks (Secrets) falhou e os jobs seguintes não prosseguiram, evidenciando o bloqueio do fluxo antes de build e deploy.

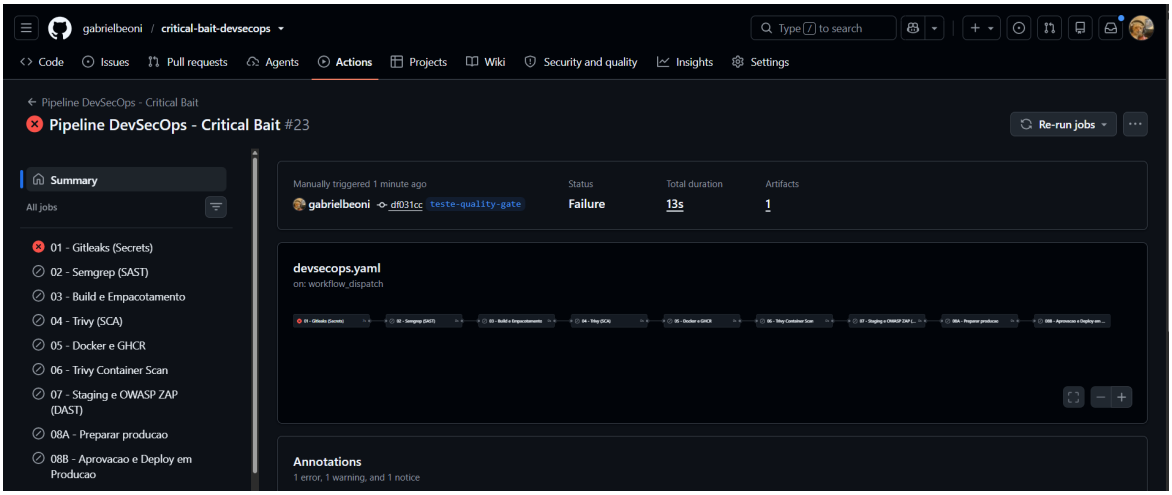


Figura 2 — Execução #23 interrompida após falha no job Gitleaks. Fonte: GitHub Actions, 2026.

Escopo da evidência: o print demonstra a falha do job e a interrupção da esteira. Como o log detalhado de detecção não está disponível, não se afirma que um segredo específico foi identificado.

6 RASTREABILIDADE DO DEPLOY

Elemento	Referência documentada
Repositório	github.com/gabrielbeoni/critical-bait-devsecops
Branch validada	main
Execução principal	GitHub Actions #15
Commit de referência	626d0a2b6bc5c75df4a87457ae7c712256314219
Imagem por SHA	ghcr.io/gabrielbeoni/critical-bait-devsecops:626d0a2b6bc5c75df4a87457ae7c712256314219
Gate de produção	Ambiente producao com aprovação manual
Destino	critical-bait-ghcr.onrender.com

A cadeia de rastreabilidade relaciona commit, tag da imagem, aprovação e destino de publicação. A tag foi observada no log do build; a comprovação do digest de publicação deve ser consultada nos artifacts e logs específicos do GHCR.

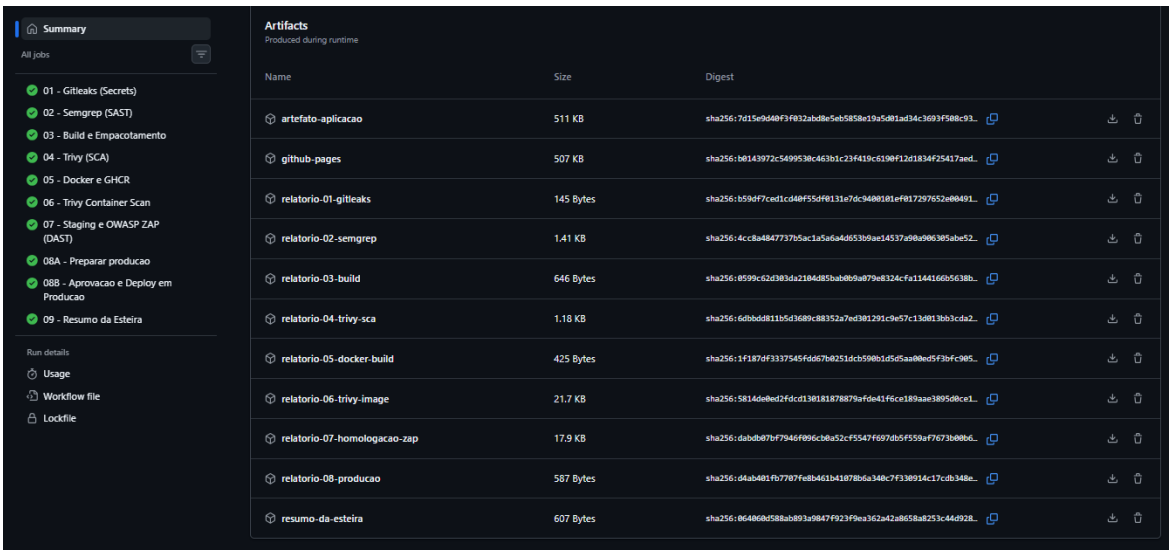


Figura 3 — Artefatos preservados na execução #15. Fonte: GitHub Actions, 2026.

7 EVIDÊNCIAS DE PRODUÇÃO

As imagens seguintes documentam o serviço publicado no Render e a aplicação acessível no navegador. A comprovação da aprovação manual integra os prints originais da entrega.

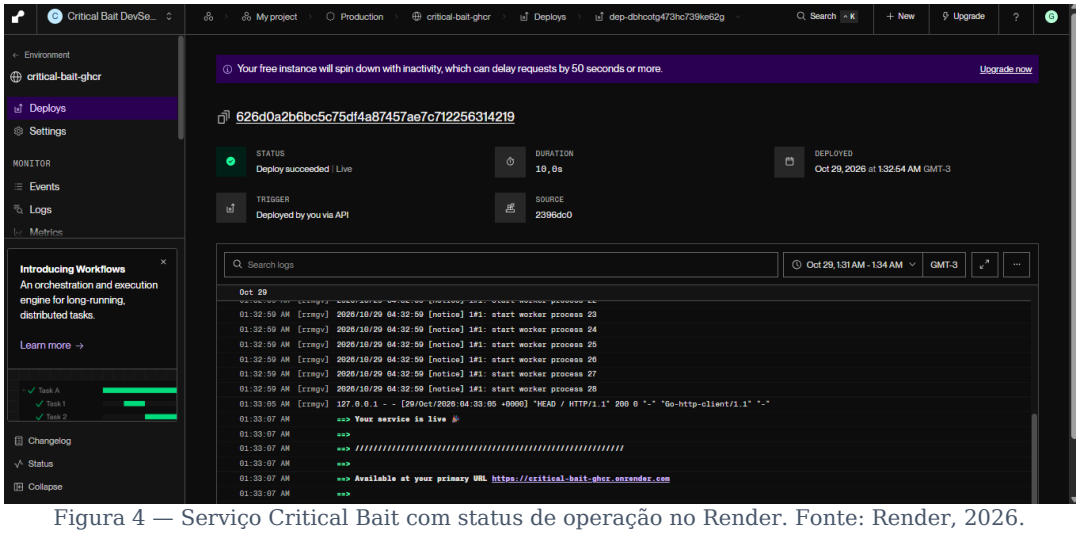


Figura 4 — Serviço Critical Bait com status de operação no Render. Fonte: Render, 2026.



Figura 5 — Aplicação Critical Bait acessível em produção. Fonte: Render, 2026.